

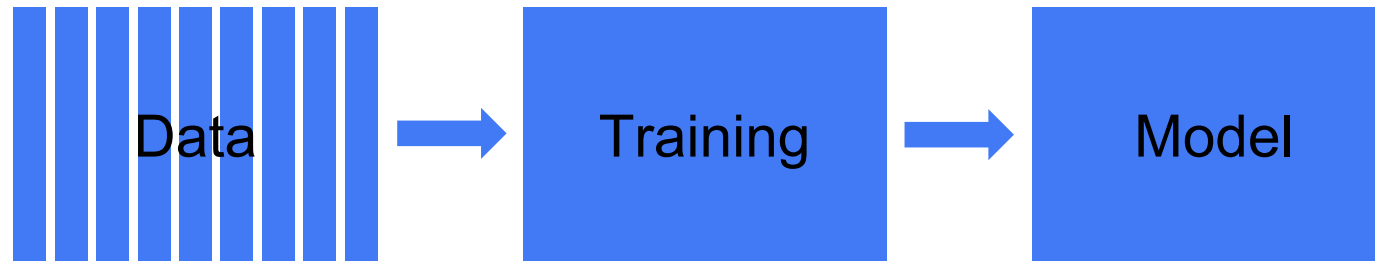


Artificial Intelligence in Interactive Systems

Online Machine Learning

Introduction into Online Machine Learning

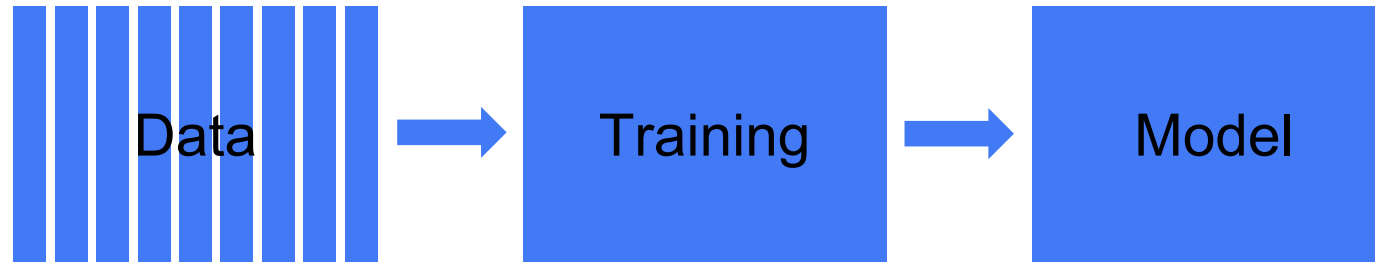
How we know Machine Learning (supervised learning) so far:



- Conventional machine learning is also often called batch learning
 - We first give the model one batch of data, learn a predictor, and then test it on unseen data
- Training and testing phases are separated

Introduction into Online Machine Learning

How we know Machine Learning (supervised learning) so far:



How Online Machine Learning works:

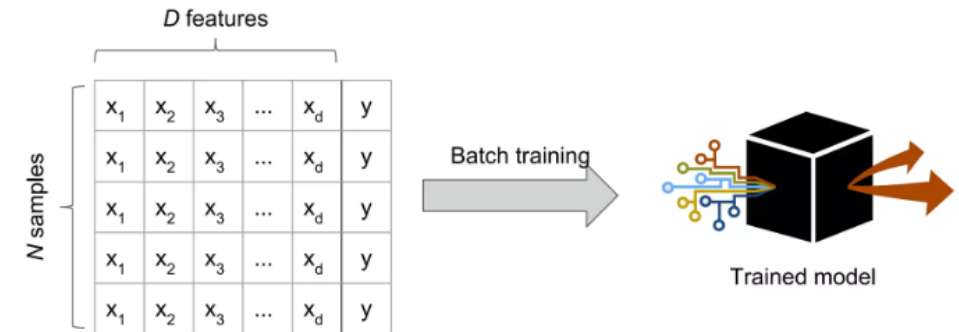


Introduction

What is Online Machine Learning?

- Models learn incrementally from a continuous stream of data
- Opposite to batch/offline learning, which trains on the entire dataset at once
 - No separate training & testing phases
 - No access to full data set at the beginning of training
 - Comparison analogous to Gradient Descent vs Stochastic Gradient Descent

Whereas in **batch (or offline) learning** you have access to the whole dataset to train on

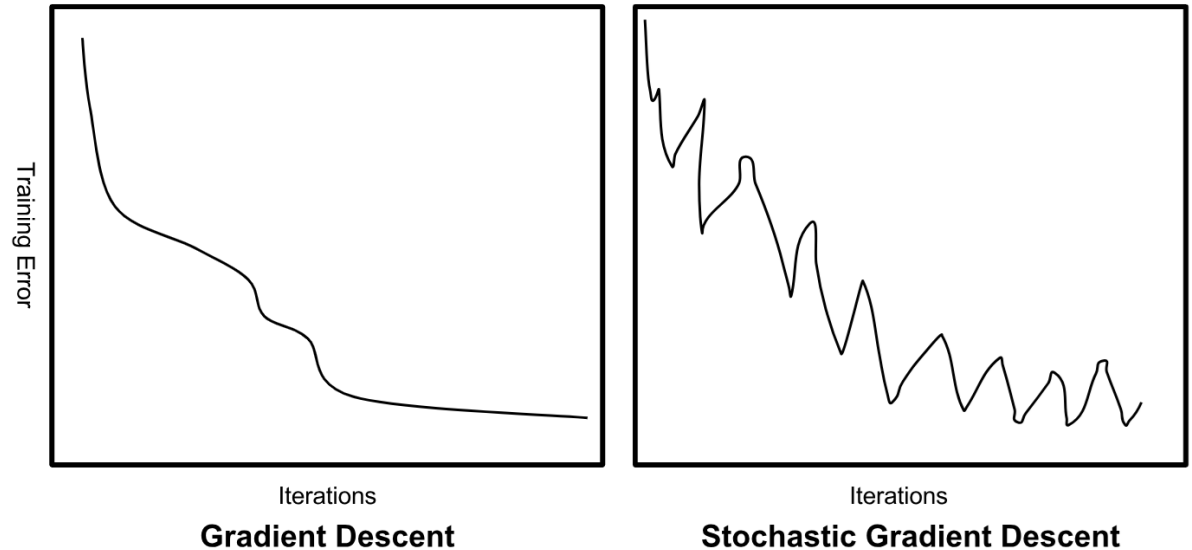


In **online learning** your model evolves as you see new data, one example at a time



Analogy: Gradient Descent vs. Stochastic Gradient Descent

- Both Gradient Descent (GD) and Stochastic Gradient Descent (SGD) are used to update a set of parameters iteratively to minimize an error function.
- **GD:** Use all samples of training data set to do one update
- **SGD:** Use a subset of training data set to do one update



https://pages.cs.wisc.edu/~spehlmann/cs760/_site/project/2017/05/04/intro.html

Also See: Lecture 05: Advanced Neural Networks – Optimizer and Hyperparameter

Key Characteristics of Online Machine Learning

- **Incremental Updates:** Models are updated continuously as new data arrives
- **Real-time Processing:** Immediate response to new information
- **Adaptability:** Ability to handle evolving data and concept drift
- **Data Efficient:** Data which has been used to train the model does not have to be stored

Use Cases and Applications

Online Machine Learning is useful in at least two scenarios:

1. When your data is too large to fit the memory
 - Usage of data generators
2. When new data is constantly being generated
 - Real-time recommendations (weather forecasting, sequential investments)
 - Anomaly Detection (fraud, spam, network attacks, new classes)
 - Adaptive Control Systems (autonomous driving)
 - IoT and Sensor Networks
 - Robotics
 - Learning new actions / unknown classes

Use Case Examples

Netflix Real-Time Recommendations [1]:

- Personalized Content Suggestions: Continuously updating recommendation algorithms based on user interactions (views, ratings, clicks).
- Dynamic Profiles: Adapting recommendations in real-time to reflect changing user preferences and behaviors.

Email Spam Detection:

- Label incoming emails as spam
- Model learns from similar features across labeled emails
- Automatically detect future spam emails which are similar to the previously labeled ones

[1] <https://research.netflix.com/research-area/recommendations>

Overview on Different Methods of Machine Learning

- **Offline/Batch Learning:** Train a model using a complete dataset, then deploy it.
- **Update Learning:** Periodically retrain the model with new data and redeploy.
 - Might be privacy concerning as full data is getting streamed between client and server
- **Federated Learning:** Perform feature engineering on local devices, send features to a server for centralized model training.
- **User Dependent Model:** Train models locally on user devices with local data.
- **Online Learning:** Continuously update the model with each new data point as it arrives.
- **Incremental Learning:** Adjust the model incrementally without retraining from scratch.

Offline Learning vs. Online Learning

	Offline Learning	Online Learning
Training	Before using the model	Continuously while using the model
Accessing Training Data	Multiple times during training	One time
Target Function	Stationary	Dynamic

Challenges with Online Machine Learning

- Concept Drift
- Catastrophic Forgetting
- Performance / Latency Trade-Off
- Ethical and Privacy Concerns

Challenges with Online Machine Learning

Concept Drift

Definition:

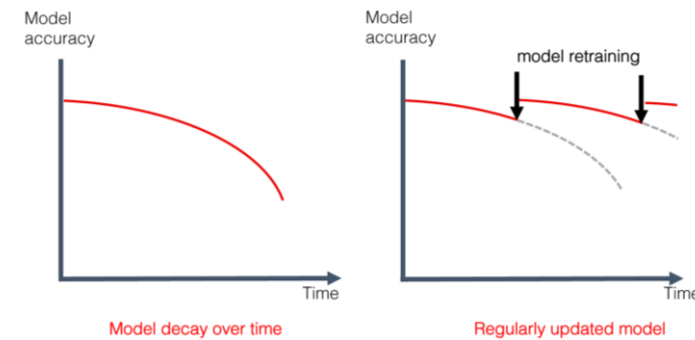
- Concept drift is when the patterns or relationships in data change over time, making a model's predictions less accurate.
- The model's quality decreases because of concept drift

Example:

- Initially, detecting spam emails might be via keywords like "free" or "win."
- Over time, spammers start using different words and tactics to avoid detection.
- The model needs to learn these new patterns to keep correctly identifying spam.

Detection and Adaptation:

- Monitor model accuracy
- Maintenance and updating
- Retrain model with newer/better data



Challenges with Online Machine Learning

Catastrophic Forgetting

Definition:

- Model loses previously learned information upon learning new information, as the parameters adapt to newer data

Example:

- Initially, the model effectively identifies spam emails using certain patterns
- As it learns to detect new spam tactics, it forgets how to recognize older spam patterns.
- Results in decreased accuracy for previously learned patterns

Detection and Adaptation:

- Check for accuracy drops of current model with older data
- Retrain model with a mix of old and new data
- Ensemble methods: Use multiple models to handle different parts of the data distribution

Challenges with Online Machine Learning

Performance / Latency Trade-Off

Latency: Delay between receiving new data and a new prediction

- Many applications require almost real-time decision making / predictions
 - Financial application, fraud detection, autonomous vehicles
- User experience suffers from high latency

Performance: Accuracy and efficiency of predictions

- Higher accuracy ensures reliable predictions
- Efficient models balance computational resources and speed → Needed for large-scale data streams

Often it is difficult to build the most accurate model with a low latency.

→ **Trade-off between complexity and speed**

- Feature engineering to reduce size of data-stream

Challenges with Online Machine Learning

Ethical and Privacy Concerns

- **Data Privacy:** Constant data collection & real-time updates risk exposing sensitive information
- **Informed Consent:** Users may be unaware that their data is being used
- **Bias and Fairness:** Can reinforce existing biases if incoming data is biased
- **Data Security:** Continuous data flow can be vulnerable to breaches, leading to unauthorized access and misuse

Mitigation Strategies:

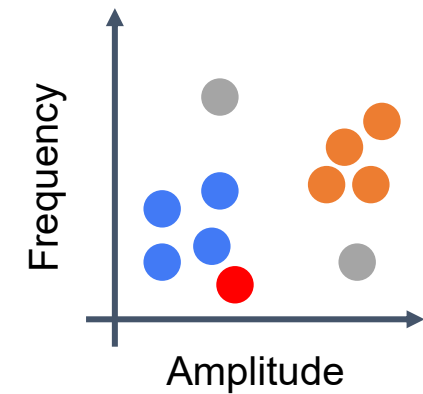
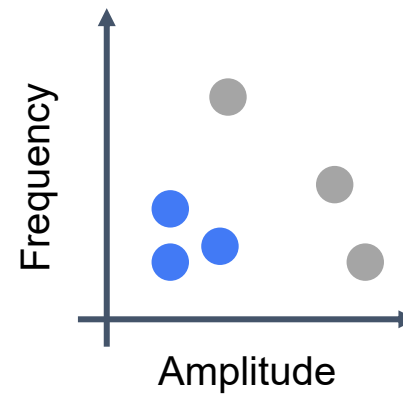
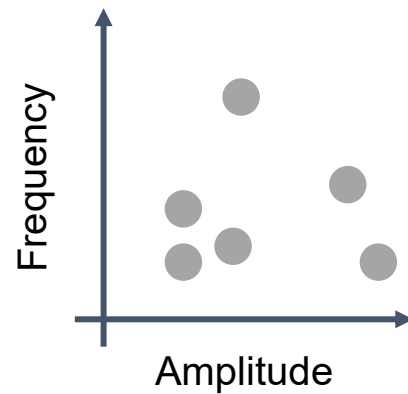
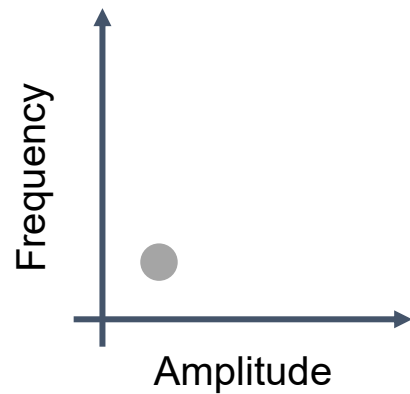
- Data anonymization
- Secure data transmission
- Federated Learning: work with features instead of raw data

Use Case: Detecting Unknown New Classes

User Dependent Model

Goal: Detect user specific new classes

Intuition: Similar to clustering from unsupervised learning



Automatic Class Discovery and One-Shot Interactions for Acoustic Activity Recognition

Automatic Class Discovery and One-Shot Interactions for Acoustic Activity Recognition

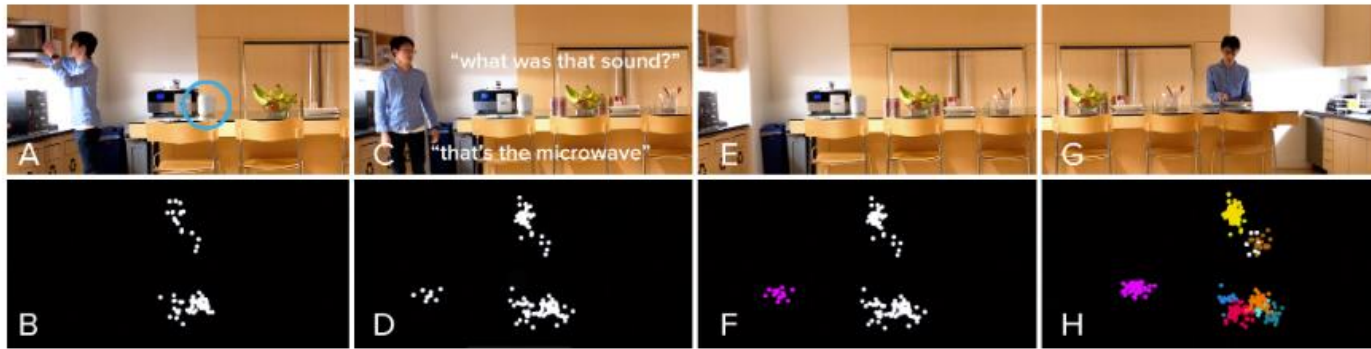


Figure 2. Listen Learner overview. A smart device is deployed in a user's environment (A). Over time, using its built-in microphone, it clusters the various sounds it hears (B and D). When the system becomes confident that a set of sounds it has heard is a singular activity, it prompts the user for a label (C). That sound cluster is then labeled (F), allowing for future recognition of that sound. Over time, the system builds up many clusters, prompting the user occasionally (G and H) allowing for a wide range of events and activities to be recognized at high accuracy.

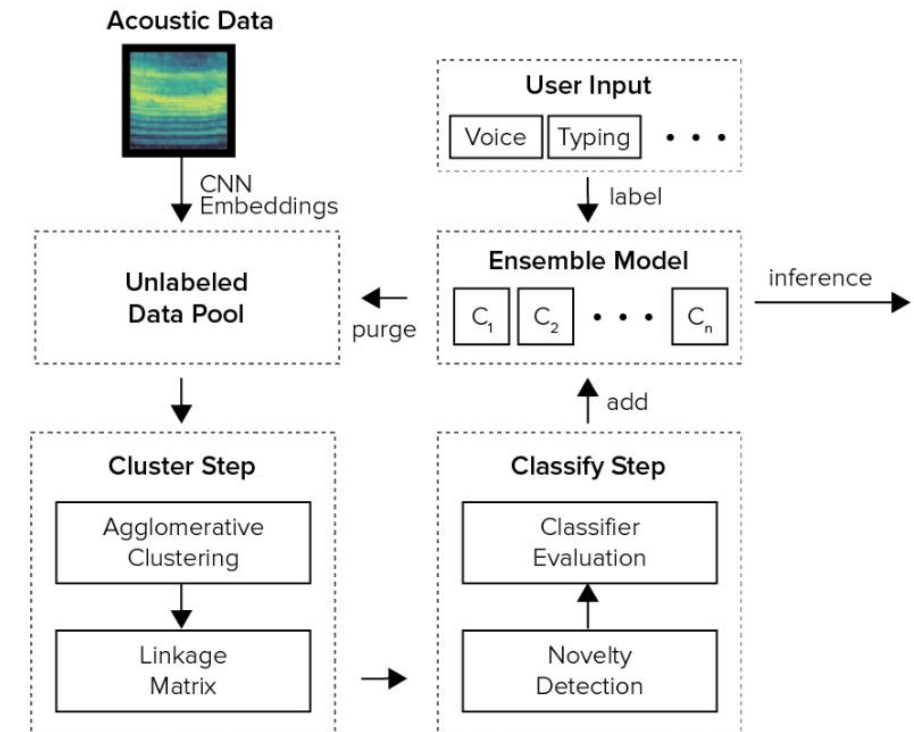
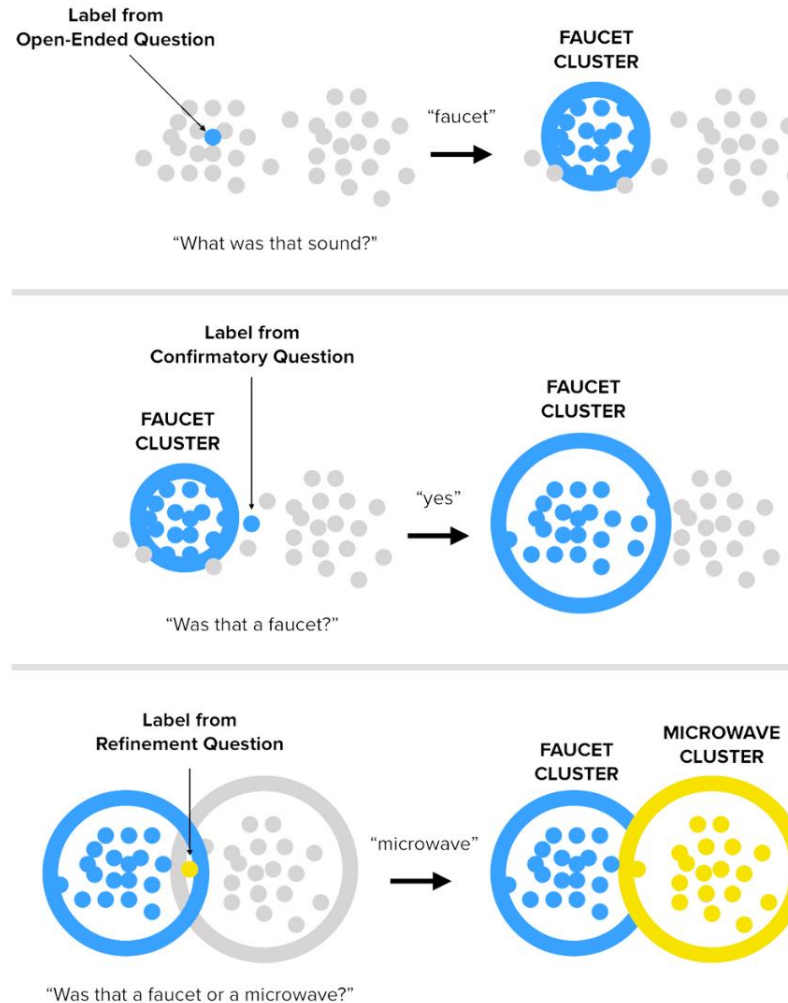


Figure 4. Listen Learner architecture and data flow.

Automatic Class Discovery and One-Shot Interactions for Acoustic Activity Recognition

User Interaction



Use Case: Detecting Unknown New Classes

Challenges

How many samples are needed to ensure stable new classes?

- Use-Case dependent
- Data-Augmentation to create multiple samples from one original sample

How to deal with higher dimensional data?

- Dimensionality reduction: PCA, T-SNE, Encoder-Decoder Models

How to ensure fixed clusters?

- Pretrained one-class classifiers

How can we build and use a generalized model from many user dependent models?

- Retrain global one-class classifiers by combining multiple samples from different users
- Use this to predict samples from new classes

Use Case: New Action Detection based on Human Pose

Combining Offline and Online Learning

Goal: Classify known human actions and detect and build classifiers for new recurring actions

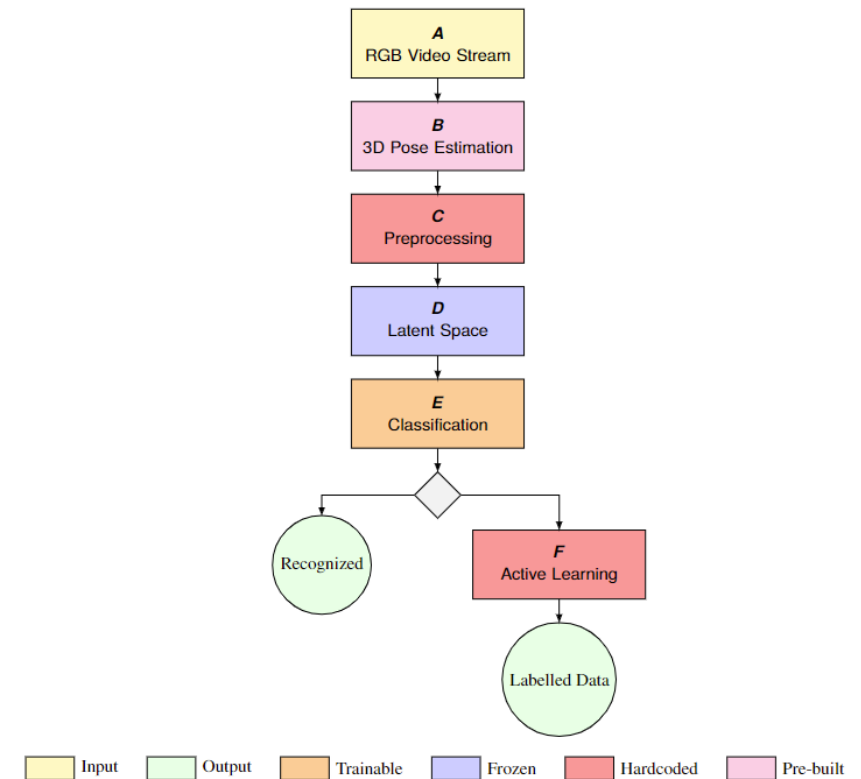
Approach: Combine offline and online machine learning

Offline:

- Use human pose as input data
- Train Encoder-Decoder Network on human poses
- Use latent pose representation as input data
- Pretrained one-class classifiers for known actions based on sequence of latent pose representations

Online: Pipe new sample through all pretrained one-class classifiers

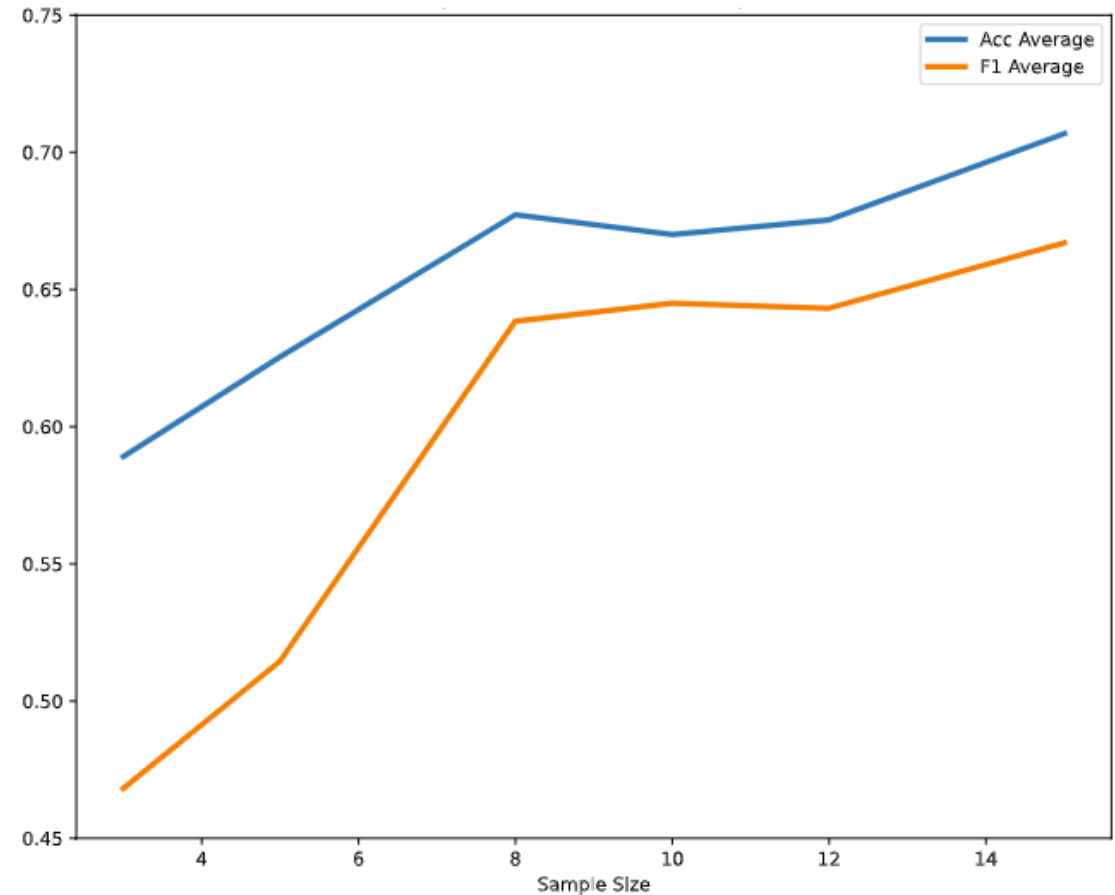
- Option A: Action is detected as known class
- Option B: Action is not detected as known class
 - Ask user for label, train new one class classifier for new actions as soon as we have multiple samples



Challenges: Detecting Unknown New Classes

How many samples are needed to ensure stable new classes?

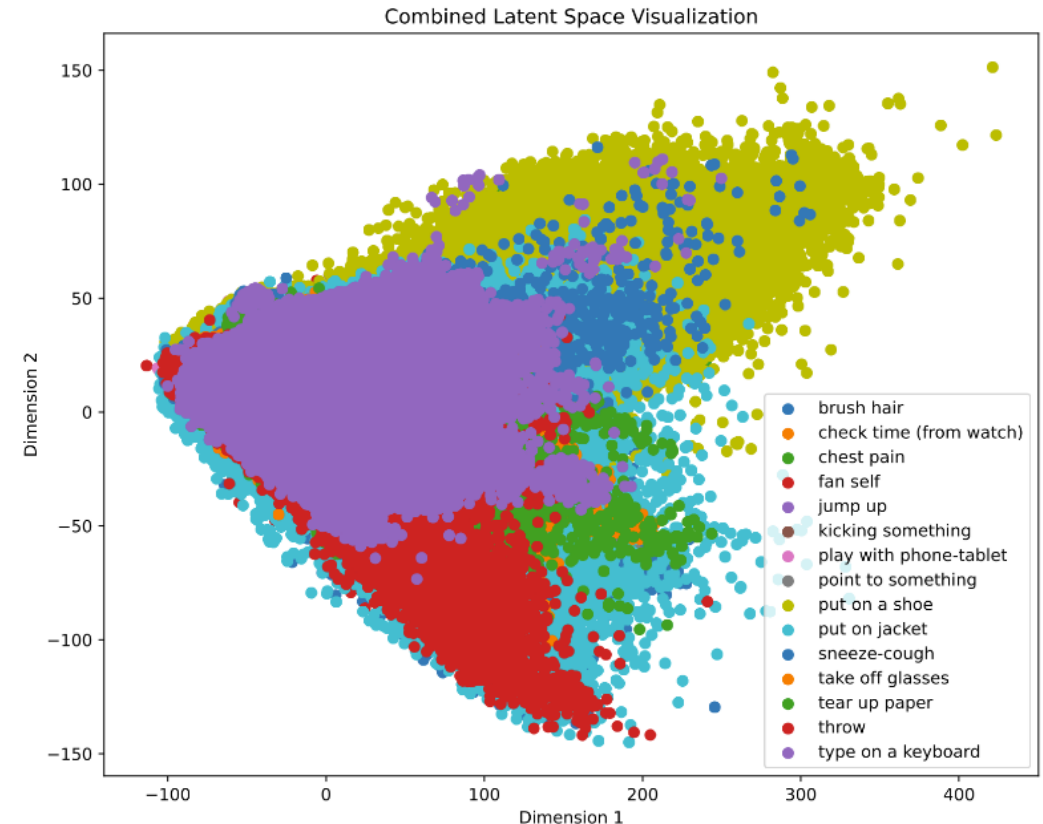
- In our human-action detection case: 8 samples lead to ~68% accuracy
- Data Augmentation can generate more samples and decrease the amount of needed original samples
- Other options: Few-Shot Learning
 - Uses pre-training knowledge for accurate predictions with few examples
 - Employs meta-learning to teach the model how to learn
 - Measures similarity using embedding-based methods



Challenges: Detecting Unknown New Classes

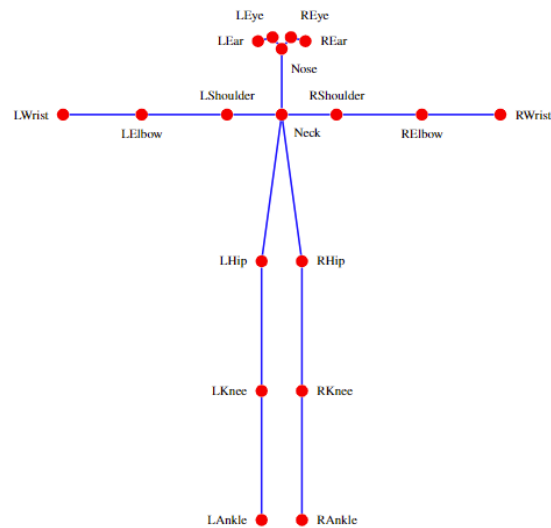
How to deal with higher dimensional data?

- Methods like PCA or T-SNE require fitting the lower dimensional space with existing data
 - Struggle with handling new data significantly different from the training data
- Utilize encoder-decoder networks for dimensionality reduction.
 - Similar approach to large language models (LLMs)
 - Train a domain-specific encoder-decoder network using relevant training data
 - Leverage latent space representations for efficient data encoding

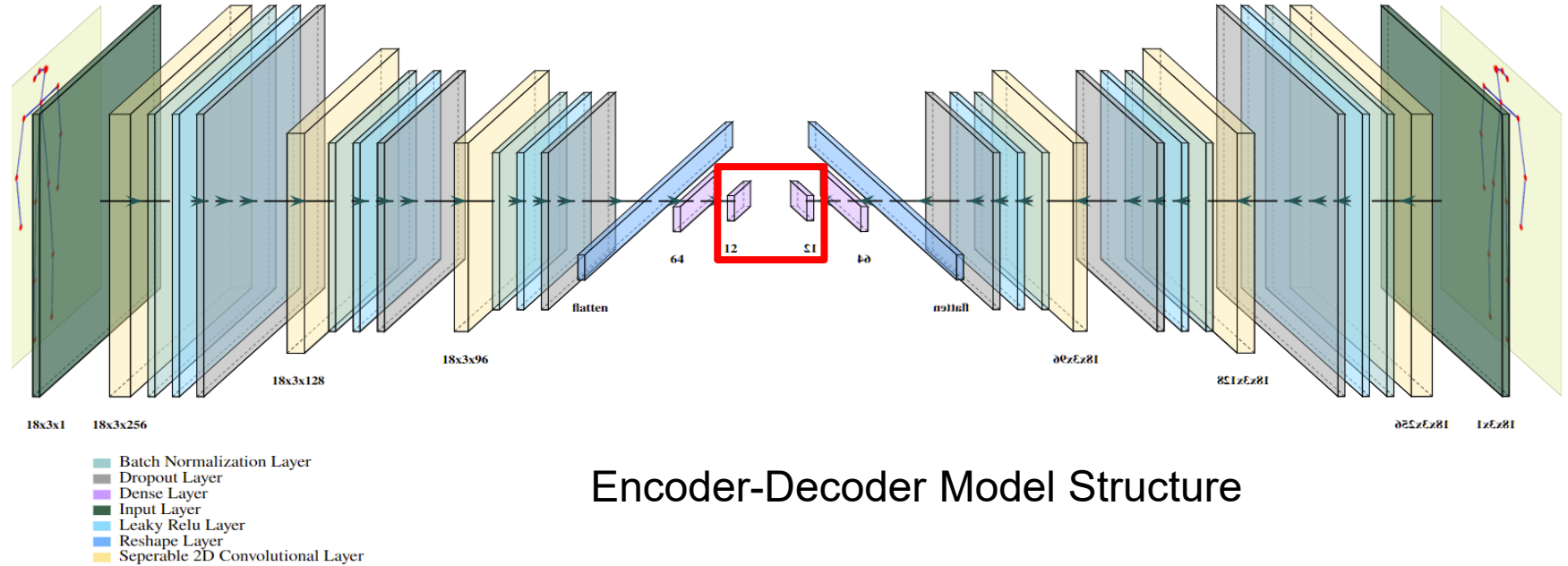


Use Case: New Action Detection based on Human Pose

Encoder-Decoder Model



Human Pose



Encoder-Decoder Model Structure

Challenges: Detecting Unknown New Classes

How to ensure fixed clusters?

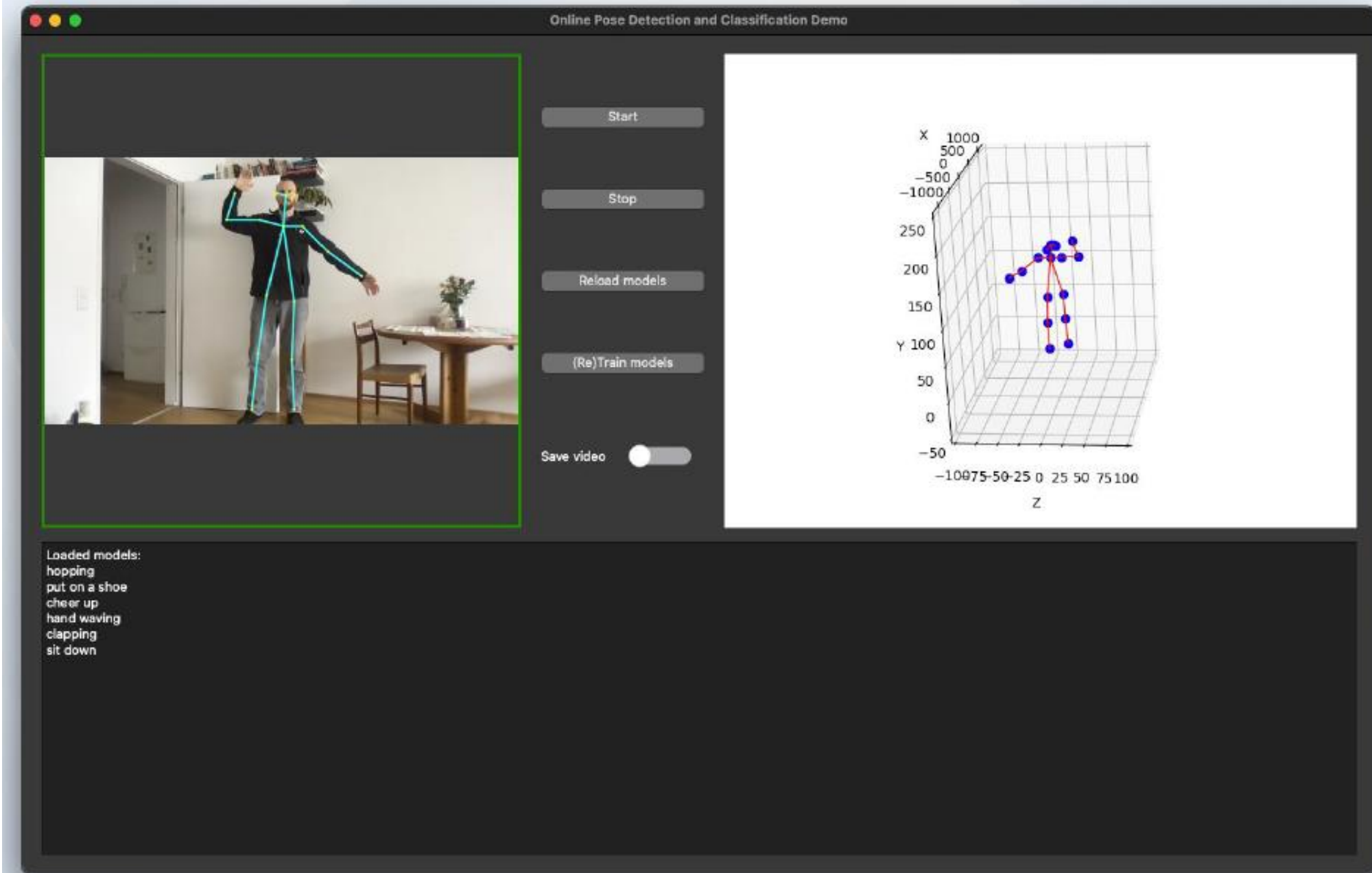
Problem:

- Ensuring distinct boundaries between known and unknown classes is crucial for accurate classification in dynamic environments

Possible Solution:

- Using many one-class classifiers, each trained to identify instances of one single action
- Implementation: SVMs, Isolation Forests, or Neural Networks





Evaluating Online Machine Learning Models

Metrics for Online Learning

- Accuracy, Precision, Recall, F1
- Time-based Metrics (e.g., Learning Curve)
- Minimize regret

Guidelines for training online machine learning models:

- Simulate real-time streaming
- Evaluate incremental updates
- Adaptive learning rate
- Test in real world scenario

Conclusion

Online Machine Learning

Online vs offline machine learning

- Online machine learning models learn incrementally from a continuous stream of data
- Comparison analogue to Gradient Descent vs Stochastic Gradient Descent

Use Cases of online machine learning

- Real-time recommendations
- Spam detection
- Anomaly detection
- Detecting unknown classes

Challenges for online machine learning

- Concept drift
- Catastrophic forgetting
- Performance & latency
- Privacy

Reading Material

- Hoi, S. C., Sahoo, D., Lu, J., & Zhao, P. (2021). Online learning: A comprehensive survey. *Neurocomputing*, 459, 249-289.
- Leon Bottou, L. (1998). Online learning and stochastic approximations. *Online learning in neural networks*, 17(9), 142.
- https://slds-lmu.github.io/i2ml/chapters/22_online_learning/

License

This file is licensed under the Creative Commons Attribution-Share Alike 4.0 (CC BY-SA) license:

<https://creativecommons.org/licenses/by-sa/4.0>

Attribution: Jan Leusmann, Sven Mayer

